

ASSIGNMENT-06

Name: Kaveti Sreenavya

HT.No: 2303A51244

Batch: 19

Task 1: AI-Based Code Completion for Conditional Eligibility Check

The task is to generate Python code that checks voting eligibility based on a person's age and citizenship status using conditional statements.

Prompt: # Generate Python code to check voting eligibility based on age and citizenship.

Code:

```
C:\Users\v1\OneDrive\Desktop\AI Coding-2026\lab06.py Additional Eligibility Check
def check_voting_eligibility(age, citizenship):
    if age >= 18 and citizenship.lower() == "yes":
        return "Eligible to vote"
    else:
        return "Not eligible to vote"

if __name__ == "__main__":
    age = int(input("Enter age: "))
    citizenship = input("Are you a citizen? (yes/no): ")
    print(check_voting_eligibility(age, citizenship))
```

Result:

```
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026> & "C:/Program Files/Python311/python.exe" "c:/Users/v1/OneDrive/Desktop/AI Coding-2026/lab06.py"
Enter age: 14
Are you a citizen? (yes/no): yes
Not eligible to vote
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026>
```

```
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026> & "C:/Program Files/Python311/python.exe" "c:/Users/v1/OneDrive/Desktop/AI Coding-2026/lab06.py"
Enter age: 18
Are you a citizen? (yes/no): yes
Eligible to vote
PS C:\Users\v1\OneDrive\Desktop\AI Coding-2026>
```

Observation:

The AI-generated code correctly applies conditional logic using logical operators. The conditions are clearly defined and easy to understand. Minor optimization was done by handling case sensitivity using the `lower()` method. The logic is accurate and suitable for real-world eligibility checks.

Task 2: AI-Based Code Completion for Loop-Based String Processing

The task is to generate Python code that counts vowels and consonants in a given string using loops.

Prompt: # Generate Python code to count vowels

and consonants in a string using a loop.

Code:

```
#Task 2: AI-Based Code Completion for Loop-Based String Processing
def count_vowels_consonants(text):
    vowels = "aeiouAEIOU"
    vowel_count = 0
    consonant_count = 0

    for char in text:
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1

    return vowel_count, consonant_count

if __name__ == "__main__":
    text = input("Enter a string: ")
    v, c = count_vowels_consonants(text)
    print("Vowels:", v)
    print("Consonants:", c)
```

Result:

```
Successfully returned - 1504
PS C:\Users\vl\OneDrive\Desktop\AI Coding-2026> & "C:/Program Files/Python311/python.exe" "c:/Users/vl/OneDrive/Desktop/AI Coding-2026
/lab06.py"
Enter a string: KAVETI SREENAVYA
Vowels: 7
Consonants: 8
PS C:\Users\vl\OneDrive\Desktop\AI Coding-2026> █
```

Observation:

The AI-generated loop efficiently iterates through each character and uses conditional checks to classify vowels and consonants. The use of `isalpha()` avoids counting spaces and symbols. The logic is correct, readable, and optimized for accuracy.

Task 3: AI-Assisted Code Completion Reflection Task

The task is to generate a complete Python program using classes, loops, and conditionals for a library management system.

Prompt: # Generate a Python program for a library management system using classes, loops, and conditional statements.

Code:

```

#Task 2: Library Management System
class Book:
    def __init__(self, title, author, isbn, available=True):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = available

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)

    def search_book(self, title):
        for book in self.books:
            if book.title.lower() == title.lower():
                return book
        return None

    def borrow_book(self, title):
        book = self.search_book(title)
        if book is None:
            return "Book not found"
        elif not book.available:
            return "Book is already borrowed"
        else:
            book.available = False
            return f"Successfully borrowed '{title}'"

    def return_book(self, title):
        book = self.search_book(title)
        if book is None:
            return "Book not found"
        elif book.available:
            return "Book was not borrowed"
        else:
            book.available = True
            return f"Successfully returned '{title}'"

    def list_available_books(self):
        available = [book for book in self.books if book.available]
        if not available:
            return "No books available"
        return [f"{book.title} by {book.author}" for book in available]

# Example usage
library = Library()
library.add_book(Book("1984", "George Orwell", "978-0451524935"))
library.add_book(Book("To Kill a Mockingbird", "Harper Lee", "978-0061120084"))
library.add_book(Book("The Great Gatsby", "F. Scott Fitzgerald", "978-0743273565"))

print(library.borrow_book("1984"))
print(library.list_available_books())
print(library.return_book("1984"))

```

Result:

```

C:\Program Files\Python\Python38\python.exe
Successfully borrowed '1984'
['To Kill a Mockingbird by Harper Lee', 'The Great Gatsby by F. Scott Fitzgerald']
Successfully returned '1984'
PS C:\Users\w\OneDrive\Desktop\AI_Coding_2026>

```

Observation:

The program allows adding books, displaying all books, and exiting the system through a menu-driven interface.

The AI-generated program effectively combines classes, loops, and conditionals. The logic is simple and functional. The menu-driven loop ensures continuous interaction. The structure is readable, and the program reflects a basic real-world library system.

Task 4: AI-Based Code Completion for Class-Based Attendance System

The task is to generate a Python class that marks and displays student attendance using loops.

Prompt: #Generate a Python class to mark and display student attendance using loops.

Code:

```
# Test the Library system
class Attendance:
    def __init__(self):
        self.attendance_record = {}

    def mark_attendance(self, student_name):
        if student_name not in self.attendance_record:
            self.attendance_record[student_name] = True # Mark present
        else:
            print(f"{student_name} is already marked present.")

    def display_attendance(self):
        print("\n--- Attendance Record ---")
        for student, present in self.attendance_record.items():
            status = "Present" if present else "Absent"
            print(f"{student}: {status}")

# Test the Attendance system
attendance = Attendance()
attendance.mark_attendance("Alice")
attendance.mark_attendance("Bob")
attendance.display_attendance()
```

Result:

```
--- Attendance Record ---
Alice: Present
Bob: Present
```

Observation

The program correctly stores and displays attendance for all students entered by the user. The AI-generated class uses a dictionary for efficient storage. Loop-based input ensures scalability. The design is simple, logical, and easy to extend for real attendance systems.

Task 5: AI-Based Code Completion for Conditional Menu Navigation

The task is to generate a Python program using loops and conditionals to simulate an ATM menu

Prompt: #Generate a Python program using loops and conditionals to simulate an ATM menu.

Code:

```

class ATM:
    def __init__(self, initial_balance=0):
        self.balance = initial_balance

    def check_balance(self):
        print(f"Your current balance: ${self.balance:.2f}")

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid amount. Please enter a positive number.")
        elif amount > self.balance:
            print("Insufficient funds.")
        else:
            self.balance -= amount
            print(f"Withdrawal successful. New balance: ${self.balance:.2f}")

    def deposit(self, amount):
        if amount <= 0:
            print("Invalid amount. Please enter a positive number.")
        else:
            self.balance += amount
            print(f"Deposit successful. New balance: ${self.balance:.2f}")

    def display_menu(self):
        while True:
            print("\n--- ATM Menu ---")
            print("1. Check Balance")
            print("2. Withdraw")
            print("3. Deposit")
            print("4. Exit")

            choice = input("Select an option (1-4): ")

            if choice == "1":
                self.check_balance()
            elif choice == "2":
                amount = float(input("Enter withdrawal amount: $"))
                self.withdraw(amount)
            elif choice == "3":
                amount = float(input("Enter deposit amount: $"))
                self.deposit(amount)
            elif choice == "4":
                print("Thank you for using ATM. Goodbye!")
                break
            else:
                print("Invalid option. Please try again.")

# Test the ATM system
atm = ATM(1000)
atm.display_menu()

```

Result:

```
--- ATM Menu ---
1. Check Balance
2. Withdraw
3. Deposit
4. Exit
Select an option (1-4): 1
Your current balance: $1000.00

--- ATM Menu ---
1. Check Balance
2. Withdraw
3. Deposit
4. Exit
Select an option (1-4): 2
Enter withdrawal amount: $500
Withdrawal successful. New balance: $500.00

--- ATM Menu ---
1. Check Balance
2. Withdraw
3. Deposit
4. Exit
Select an option (1-4): 3
Enter deposit amount: $1200
Deposit successful. New balance: $1700.00
```

```
--- ATM Menu ---
1. Check Balance
2. Withdraw
3. Deposit
4. Exit
Select an option (1-4): 4
Thank you for using ATM. Goodbye!
PS C:\Users\vl\OneDrive\Desktop\AI Coding-2026> █
```

Observation:

The ATM menu allows checking balance, withdrawing, depositing, and exiting correctly. The AI-generated logic correctly uses loops for repeated menu display and conditionals for option handling. The program is efficient and user-friendly. Minor improvements such as input validation could further enhance robustness.

